



Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ _____ «Информатика и системы управления»

КАФЕДРА _____ «Теоретическая информатика и компьютерные технологии»

Лабораторная работа № 5

по курсу «Алгоритмы компьютерной графики»

Студент группы ИУ9-42Б Федуков А. А.

Преподаватель: Цалкович П.А.

17 апреля 2025 г.

Цель работы

- Реализовать один из алгоритмов отсечения определенного типа в пространстве заданной размерности (в соответствии с вариантом):
- Ввод исходных данных каждого из алгоритмов производится интерактивно с помощью клавиатуры и/или мыши.

Мой вариант:

Алгоритм отсечения	Размерность пространства отсечения	Тип отсечения
Кируса-Бека	Трёхмерное	Внешнее

Теория

Алгоритм Кируса–Бека (Cyrus–Beck) представляет собой параметрический метод отсечения отрезков, предназначенный для работы с выпуклыми отсекателями.

Отрезок определяется двумя точками P_0 и P_1 и представляется в параметрической форме:

$$P(t) = P_0 + t(P_1 - P_0), \quad t \in [0, 1],$$

где t — параметр, определяющий положение точки на отрезке.

Каждая грань отсекателя в трёхмерном пространстве задаётся нормалью $\vec{n}$ и расстоянием d от начала координат до плоскости грани. Для каждой грани определяется значение параметра t , при котором отрезок пересекает соответствующую плоскость. Это значение вычисляется по формуле:

$$t = \frac{d - \vec{n} \cdot P_0}{\vec{n} \cdot (P_1 - P_0)}.$$

В процессе работы алгоритма поддерживаются два параметра: t_{enter} и t_{exit} , которые инициализируются значениями 0 и 1 соответственно. Эти параметры обновляются при прохождении через каждую грань отсекателя:

- Если $\vec{n} \cdot (P_1 - P_0) < 0$, то отрезок входит в отсекатель, и $t_{\text{enter}} = \max(t_{\text{enter}}, t)$.

- Если $\vec{n} \cdot (P_1 - P_0) > 0$, то отрезок выходит из отсекаателя, и $t_{\text{exit}} = \min(t_{\text{exit}}, t)$.
- Если знаменатель равен нулю, то отрезок параллелен плоскости грани; в этом случае он может быть полностью вне отсекаателя.

После обработки всех граней отсекаателя, если условие $t_{\text{enter}} \leq t_{\text{exit}}$ выполняется, отрезок частично проходит через область отсечения, и отсечённый фрагмент можно определить как:

$$P_{\text{clipped}} = [P(t_{\text{enter}}), P(t_{\text{exit}})].$$

Реализация

Используя GLFW для управления окном и ввода, а также OpenGL для визуализации, по заданным в пространстве точкам я отрисовываю отрезок. Отсекатель представлен в виде шести граней куба, каждая из которых задаётся нормалью и расстоянием до центра координат. Для каждой грани вычисляется параметр пересечения с отрезком и соответствующим образом обновляются значения t_{enter} и t_{exit} .

Отрезок можно двигать цифрами клавиатуры 1-0. Если отрезок пересекает область отсечения (часть куба, заданного изначально для удобства), то отображается его отсечённая часть. Исходный отрезок — красный, отсечённый фрагмент — зелёный, точки входа и выхода — жёлтая и голубая соответственно. Также реализована возможность переключения между каркасным и полупрозрачным режимами отображения куба. по клавише W

Код

Листинг 1: Файл main.py

```
1 import glfw
2 from OpenGL.GL import *
3 from OpenGL.GLU import *
4 import numpy as np
5
6 # Initialize GLFW
```

```

7  if not glfw.init():
8      raise Exception("GLFW initialization failed")
9
10 width, height = 800, 600
11 window = glfw.create_window(width, height, "3D Cyrus-Beck Clipping",
    None, None)
12 if not window:
13     glfw.terminate()
14     raise Exception("GLFW window creation failed")
15
16 glfw.make_context_current(window)
17 glEnable(GL_DEPTH_TEST)
18 glEnable(GL_BLEND)
19 glBlendFunc(GL_SRC_ALPHA, GL_ONE_MINUS_SRC_ALPHA)
20
21 # Settings
22 wireframe_mode = False
23 cube_size = 0.8
24 zoom = 5.0
25 rotation_x = 30.0
26 rotation_y = 30.0
27
28 # Line parameters
29 line_start = np.array([-1.5, -1.5, -1.5], dtype=np.float32)
30 line_end = np.array([1.5, 1.5, 1.5], dtype=np.float32)
31
32 def setup_projection():
33     glMatrixMode(GL_PROJECTION)
34     glLoadIdentity()
35     gluPerspective(45, width/height, 0.1, 100.0)
36     glMatrixMode(GL_MODELVIEW)
37
38 def cyrus_beck_clip(P0, P1):
39     # Define planes with correct distances (d) for a cube centered at
    the origin
40     planes = [
41         (np.array([0, 0, 1], dtype=np.float32), cube_size), # Front
42         (np.array([0, 0, -1], dtype=np.float32), cube_size), # Back
43         (np.array([0, 1, 0], dtype=np.float32), cube_size), # Top
44         (np.array([0, -1, 0], dtype=np.float32), cube_size), # Bottom
45         (np.array([1, 0, 0], dtype=np.float32), cube_size), # Right
46         (np.array([-1, 0, 0], dtype=np.float32), cube_size) # Left
47     ]
48
49     t_enter = 0.0
50     t_exit = 1.0

```

```

51 D = P1 - P0 # Direction vector of the line
52
53 for normal, d in planes:
54     numerator = d - np.dot(normal, P0)
55     denominator = np.dot(normal, D)
56
57     if abs(denominator) < 1e-6: # Line is parallel to the plane
58         if numerator < 0: # Line is outside the plane
59             return None
60     else:
61         t = numerator / denominator
62         if denominator < 0: # Entering the plane
63             t_enter = max(t_enter, t)
64         else: # Exiting the plane
65             t_exit = min(t_exit, t)
66
67         if t_enter > t_exit: # No intersection
68             return None
69
70 # Calculate the clipped points if valid t values exist
71 if 0 <= t_enter <= 1 and 0 <= t_exit <= 1:
72     return (P0 + t_enter * D, P0 + t_exit * D)
73 return None
74
75 def draw_cube():
76     vertices = [
77         [-cube_size, -cube_size, -cube_size],
78         [cube_size, -cube_size, -cube_size],
79         [cube_size, cube_size, -cube_size],
80         [-cube_size, cube_size, -cube_size],
81         [-cube_size, -cube_size, cube_size],
82         [cube_size, -cube_size, cube_size],
83         [cube_size, cube_size, cube_size],
84         [-cube_size, cube_size, cube_size]
85     ]
86
87     faces = [
88         [0, 1, 2, 3], [4, 5, 6, 7], [0, 1, 5, 4],
89         [2, 3, 7, 6], [0, 3, 7, 4], [1, 2, 6, 5]
90     ]
91
92     if wireframe_mode:
93         glColor3f(0.3, 0.3, 0.3)
94         glPolygonMode(GL_FRONT_AND_BACK, GL_LINE)
95     else:
96         glColor4f(0.7, 0.7, 0.7, 0.3)

```

```

97         glPolygonMode(GL_FRONT_AND_BACK, GL_FILL)
98
99     glBegin(GL_QUADS)
100     for face in faces:
101         for vertex in face:
102             glVertex3fv(vertices[vertex])
103     glEnd()
104
105     # Always draw wireframe over solid
106     if not wireframe_mode:
107         glColor3f(0.3, 0.3, 0.3)
108         glPolygonMode(GL_FRONT_AND_BACK, GL_LINE)
109         glBegin(GL_QUADS)
110         for face in faces:
111             for vertex in face:
112                 glVertex3fv(vertices[vertex])
113         glEnd()
114         glPolygonMode(GL_FRONT_AND_BACK, GL_FILL)
115
116     def draw_line(start, end, color, width=1.0):
117         glColor3fv(color)
118         glLineWidth(width)
119         glBegin(GL_LINES)
120         glVertex3fv(start)
121         glVertex3fv(end)
122         glEnd()
123
124     def display():
125         glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT)
126         glLoadIdentity()
127         gluLookAt(0, 0, zoom, 0, 0, 0, 0, 1, 0)
128         glRotatef(rotation_x, 1, 0, 0)
129         glRotatef(rotation_y, 0, 1, 0)
130
131         # Draw axes
132         glBegin(GL_LINES)
133         glColor3f(1,0,0); glVertex3f(0,0,0); glVertex3f(1,0,0)
134         glColor3f(0,1,0); glVertex3f(0,0,0); glVertex3f(0,1,0)
135         glColor3f(0,0,1); glVertex3f(0,0,0); glVertex3f(0,0,1)
136         glEnd()
137
138         draw_cube()
139
140         # Draw original line
141         draw_line(line_start, line_end, (1, 0, 0))
142

```

```

143 # Clip and draw visible segment
144 clipped = cyrus_beck_clip(line_start, line_end)
145 if clipped:
146     # Draw thick green line for clipped segment
147     draw_line(clipped[0], clipped[1], (0, 1, 0), 4.0)
148
149     # Draw markers at clip points
150     glPointSize(10.0)
151     glBegin(GL_POINTS)
152     glColor3f(1,1,0); glVertex3fv(clipped[0]) # Yellow entry point
153     glColor3f(0,1,1); glVertex3fv(clipped[1]) # Cyan exit point
154     glEnd()
155
156     glfw.swap_buffers(window)
157
158 def key_callback(window, key, scancode, action, mods):
159     global rotation_x, rotation_y, zoom, line_start, line_end,
160     wireframe_mode
161
162     if action == glfw.PRESS or action == glfw.REPEAT:
163         # Rotation
164         if key == glfw.KEY_LEFT: rotation_y -= 5
165         elif key == glfw.KEY_RIGHT: rotation_y += 5
166         elif key == glfw.KEY_UP: rotation_x -= 5
167         elif key == glfw.KEY_DOWN: rotation_x += 5
168         # Zoom
169         elif key == glfw.KEY_Z: zoom -= 0.5
170         elif key == glfw.KEY_X: zoom += 0.5
171         # Reset view
172         elif key == glfw.KEY_C:
173             rotation_x, rotation_y, zoom = 30.0, 30.0, 5.0
174         # Line movement
175         elif key == glfw.KEY_1: line_start[0] -= 0.1
176         elif key == glfw.KEY_2: line_start[0] += 0.1
177         elif key == glfw.KEY_3: line_start[1] -= 0.1
178         elif key == glfw.KEY_4: line_start[1] += 0.1
179         elif key == glfw.KEY_5: line_start[2] -= 0.1
180         elif key == glfw.KEY_6: line_start[2] += 0.1
181         elif key == glfw.KEY_7: line_end[0] -= 0.1
182         elif key == glfw.KEY_8: line_end[0] += 0.1
183         elif key == glfw.KEY_9: line_end[1] -= 0.1
184         elif key == glfw.KEY_0: line_end[1] += 0.1
185         elif key == glfw.KEY_MINUS: line_end[2] -= 0.1
186         elif key == glfw.KEY_EQUAL: line_end[2] += 0.1
187         # Reset line
188         elif key == glfw.KEY_R:

```

```

188         line_start = np.array([-1.5, -1.5, -1.5], dtype=np.float32)
189         line_end = np.array([1.5, 1.5, 1.5], dtype=np.float32)
190     # Toggle wireframe
191     elif key == glfw.KEY_W:
192         wireframe_mode = not wireframe_mode
193
194 glfw.set_key_callback(window, key_callback)
195 setup_projection()
196
197 while not glfw.window_should_close(window):
198     display()
199     glfw.poll_events()
200
201 glfw.terminate()

```

Вывод программы

Программа создала окно, отрисовала куб и линию. Перечения куба и линии выражены при помощи цветовой индикации. При движении линии индикация соответственно меняется в зависимости от площади отсечения.

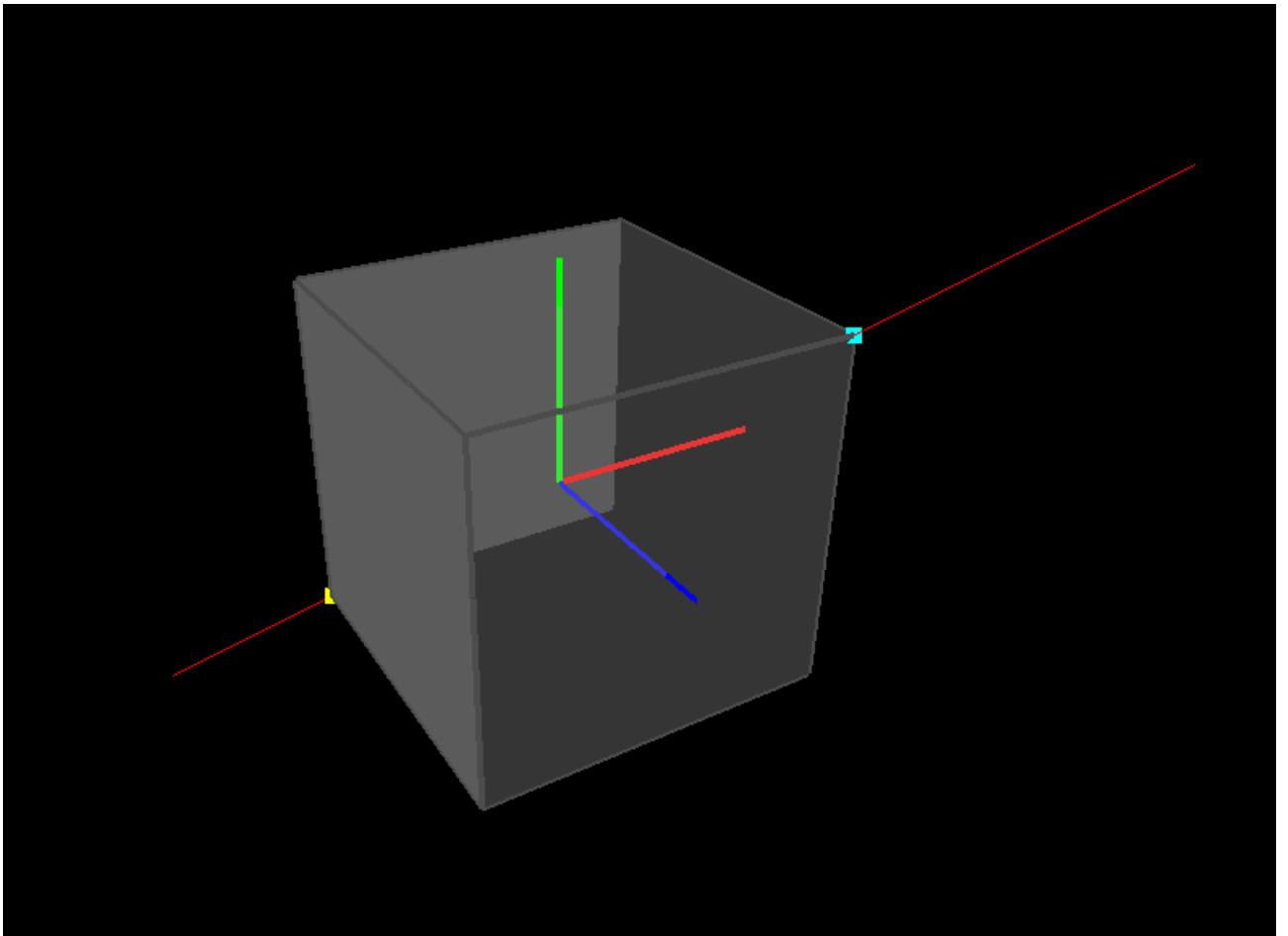


Рис. 1 — Исходная фигура

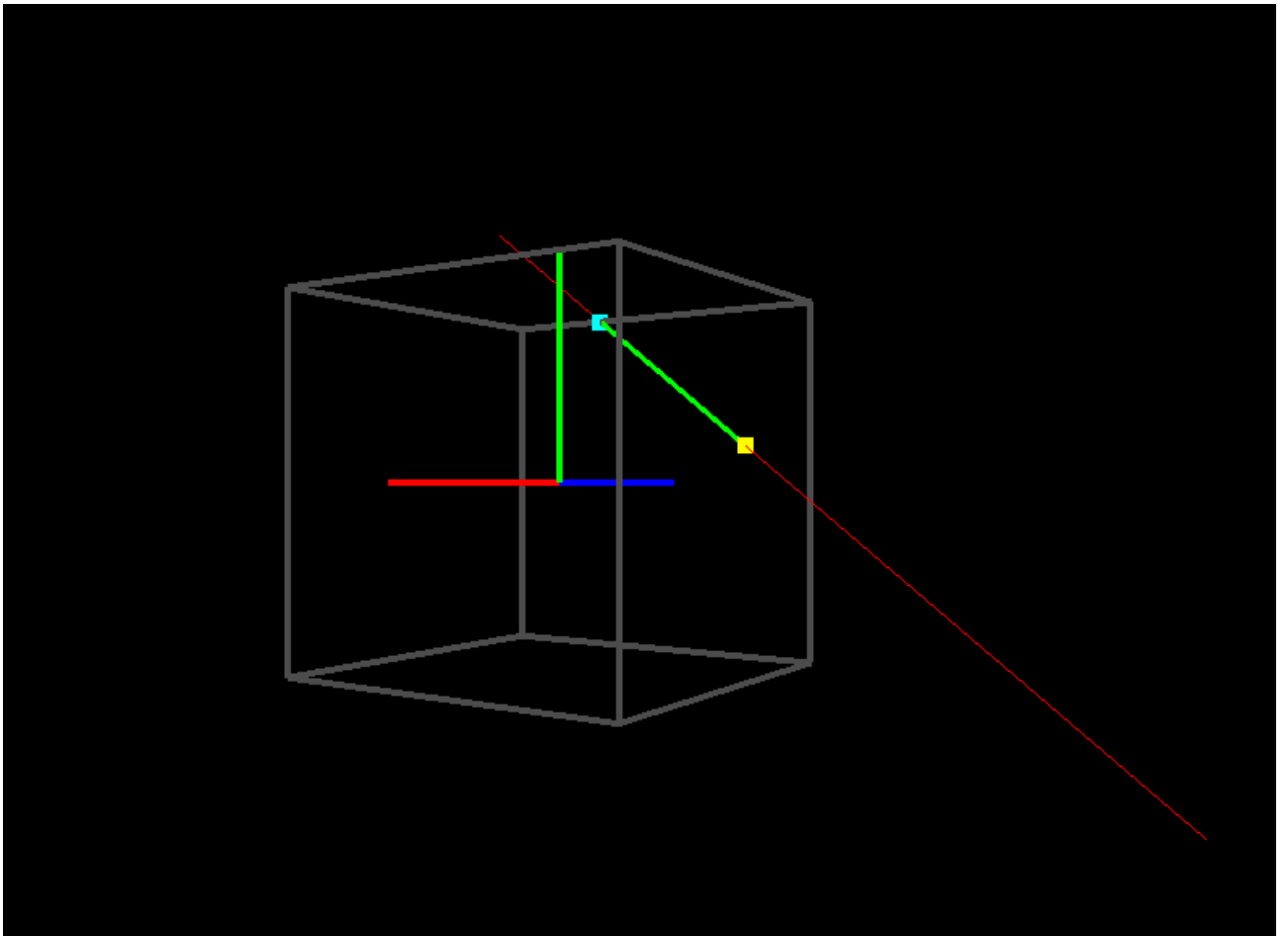


Рис. 2 — Фигура после передвижения камеры и линии

Вывод

В ходе выполнения лабораторной работы был изучен и реализован алгоритм Кируса–Бека для трёхмерного внешнего отсечения отрезков. Полученная программа позволяет интерактивно управлять положением отрезка и визуализировать результат его отсечения выпуклым отсекателем. Работа способствовала пониманию принципов геометрических алгоритмов в трёхмерном пространстве и закреплению навыков работы с OpenGL.